

---

# De la classification des chiffres manuscrits à la détection de cancers du sein

Introduction aux réseaux de neurones convolutifs

---

**Promotion :** 2028

**Équipe :** 6

**Étudiants :**

Alexandre LE GALLIARD

Taino MOUTAMALE

Lucie LE SAUX

Baptiste NAUCHE

Malik LAHRICHI

# 1 Classification de chiffres manuscrits (MNIST)

## 1.1 Jeu de données et prétraitement

Le jeu de données **MNIST** contient 70 000 images de chiffres manuscrits en niveaux de gris, dont 60 000 pour l'entraînement et 10 000 pour le test. Chaque image est de taille  $28 \times 28$  pixels et est aplatie en un vecteur  $\vec{x} \in R^{784}$ , puis normalisée dans l'intervalle  $[0, 1]$ . Les étiquettes  $y \in \{0, \dots, 9\}$  sont encodées en one-hot  $Y \in \{0, 1\}^{n \times 10}$  afin d'utiliser la fonction de coût d'entropie croisée.

## 1.2 Modèle linéaire (softmax)

Nous commençons par un classificateur linéaire multi-classe appliqué aux vecteurs d'images  $\vec{x} \in R^{784}$ . Pour chaque image, le modèle calcule un vecteur de scores

$$\vec{o} = A\vec{x} + \vec{b}, \quad A \in R^{10 \times 784}, \quad \vec{b} \in R^{10},$$

où la ligne  $k$  de  $A$  contient les poids associés à la classe  $k$  (figure 10). Les scores sont transformés en probabilités par la fonction softmax (figure 11) :

$$P_k(\vec{x}) = \frac{e^{o_k}}{\sum_{j=0}^9 e^{o_j}}.$$

La prédiction finale (figure 12) est obtenue en prenant la classe avec la probabilité maximale,  $\hat{y} = \arg \max_k P_k(\vec{x})$ .

L'entraînement minimise l'entropie croisée entre les probabilités prédites  $P$  et les étiquettes one-hot  $Y$  (figure 13) :

$$\mathcal{L}(A, \vec{b}) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^9 y_i^{(k)} \ln P_k(\vec{x}_i).$$

En dérivant cette expression, on obtient le gradient

$$\frac{\partial \mathcal{L}}{\partial A} = \frac{1}{n} (P - Y)^\top X, \quad \frac{\partial \mathcal{L}}{\partial \vec{b}} = \frac{1}{n} \sum_{i=1}^n (P_i - Y_i),$$

ce qui permet d'implémenter une descente de gradient mini-batch (figure 14). Pour évaluer le modèle, nous comparons les prédictions  $\hat{y}$  aux vraies étiquettes  $y$  sur le jeu de test ; la précision (accuracy) est définie comme la proportion d'exemples correctement classés (figure 15). Avec 7 850 paramètres, ce modèle atteint une précision d'environ 92,3 %, soit un taux d'erreur d'environ **7,7 %** sur le jeu de test : il constitue une première baseline simple et interprétable pour la suite de l'étude.

## 1.3 Perceptron multi-couches (MLP)

Pour capturer des relations non linéaires entre les pixels, nous ajoutons des couches cachées avec activation ReLU. Nous testons deux architectures :

$$784 \rightarrow 128 \rightarrow 10, \quad 784 \rightarrow 128 \rightarrow 64 \rightarrow 10,$$

suivies d'une couche softmax en sortie.

**Justification du choix 128 et 64.** Les tailles  $128 = 2^7$  et  $64 = 2^6$  résultent d'un compromis entre capacité de représentation, coût de calcul et risque de surapprentissage. Ce sont des **puissances de 2**, classiques en deep learning car elles s'alignent naturellement sur la taille des batchs et l'organisation mémoire des GPU. Elles offrent une **réduction progressive de la dimension** (de 784 à 128 puis 64, soit un facteur de compression  $\sim 6$  puis 2), qui force le réseau à construire des représentations de plus en plus abstraites sans goulot d'étranglement brutal. Le MLP à deux couches contient alors environ 109 000 paramètres, soit de l'ordre de deux fois la taille du jeu d'entraînement : suffisant pour modéliser les non-linéarités tout en limitant le surapprentissage et en restant entraînable sur CPU en quelques minutes. Des tailles plus grandes (256, 512) sont possibles mais apportent peu sur MNIST, tandis que des tailles trop petites (32, 16) sous-dimensionnent la première couche.

La propagation avant est résumée figure 16 : chaque couche cachée applique une transformation linéaire suivie de l'activation ReLU. La rétropropagation (figure 17) calcule les gradients couche par couche : l'erreur ( $P - Y$ ) est propagée à la dernière couche, puis aux couches cachées en la multipliant par les matrices de poids et par la dérivée de l'activation. L'entraînement est réalisé par descente de gradient mini-batch, avec la même fonction de coût (entropie croisée) que pour le modèle linéaire. Le tableau 1 résume les performances obtenues : l'ajout d'une première couche cachée réduit fortement l'erreur (de 7,72 % à 3,46 %), tandis que la seconde couche apporte un gain plus modeste.

## 1.4 Études complémentaires : profondeur, normalisation, activations

Pour répondre à plusieurs questions soulevées lors de la soutenance, nous avons réalisé trois études de sensibilité autour du MLP de référence  $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ . Toutes les expériences utilisent les mêmes hyperparamètres (Adam,  $\text{lr} = 10^{-3}$ , batch 128, 10 époques, graine 42).

**Ajout d'une troisième couche cachée.** Nous avons comparé l'architecture à deux couches cachées avec une variante à trois couches  $784 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$ , ajoutant environ 2 400 paramètres supplémentaires. Les résultats sont : 109 386 paramètres pour 3,27 % d'erreur vs 111 818 pour 3,21 %. Le gain apporté par la troisième couche est marginal ( $\approx 0,06$  point), et de l'ordre de la variance d'une exécution à l'autre. Sur MNIST, la frontière de décision est suffisamment simple pour qu'une deuxième couche cachée suffise : ajouter de la profondeur augmente la capacité du réseau, mais aussi le risque de surapprentissage et le coût d'entraînement, sans bénéfice mesurable. La profondeur devient bien plus utile sur des jeux d'images naturelles (CIFAR-10, ImageNet), où les structures à reconnaître sont hiérarchiques.

**La normalisation est-elle indispensable ?** La normalisation des pixels dans  $[0, 1]$  (division par 255) joue un rôle clé pour la stabilité de l'entraînement. Réentraîné sans normalisation (pixels bruts dans  $[0, 255]$ ), le MLP devient instable et l'erreur de test dépasse 30 % contre 3,27 % avec normalisation. Sans normalisation, les scores  $\vec{o} = A\vec{x} + \vec{b}$  prennent des valeurs très grandes, ce qui provoque une saturation de la softmax et des gradients soit nuls soit divergents. La normalisation n'est pas indispensable en théorie (on pourrait compenser en diminuant fortement le taux d'apprentissage), mais elle est en pratique *quasi-obligatoire* pour obtenir un entraînement rapide et stable. Toute fonction qui ramène les entrées dans un intervalle réduit autour de 0 joue ce rôle : division

par 255, standardisation  $(x - \mu)/\sigma$ , ou batch-normalisation à l'intérieur du réseau.

**Comparaison des fonctions d'activation.** Nous avons comparé quatre activations usuelles sur le MLP de référence, en conservant la softmax en sortie : Sigmoid (5,10 %), tanh (3,80 %), ReLU (3,27 %) et Leaky ReLU (3,25 %). Les deux activations historiques (sigmoid et tanh) sont saturantes : leurs dérivées tendent vers 0 pour des entrées de grande valeur absolue, ce qui ralentit fortement la rétropropagation des gradients dans les couches profondes (*vanishing gradient*). La fonction **ReLU** ( $\max(0, x)$ ) est non saturante du côté positif et possède une dérivée constante égale à 1, ce qui accélère la convergence et permet de gagner  $\approx 1,8$  point par rapport à la sigmoid. La variante **Leaky ReLU** corrige le problème des « neurones morts » en laissant passer une petite pente  $\alpha x$  pour les entrées négatives, mais sur MNIST les gains sont négligeables. C'est pour ces raisons que ReLU s'est imposée comme l'activation par défaut dans la suite du projet.

## 1.5 Évaluation et analyse des erreurs

Pour comparer le modèle linéaire et les MLP, nous calculons les taux d'erreur sur l'ensemble d'entraînement et sur l'ensemble de test. Les modèles multi-couches obtiennent des erreurs train légèrement plus faibles que le modèle linéaire, ce qui traduit une capacité de représentation plus élevée. Sur le test, l'erreur passe d'environ 7,7 % pour le modèle linéaire à 3,5 % puis 3,3 % pour les MLP à une et deux couches cachées.

Les chiffres mal classés par le meilleur modèle (MLP à deux couches) concernent principalement des paires ambiguës (4/9, 3/5, 2/7), pour lesquelles la forme manuscrite est proche. De nombreuses erreurs proviennent d'écritures atypiques : chiffres inclinés, traits non fermés, proportions inhabituelles, qui s'écartent des exemples « prototypes » vus pendant l'entraînement. La matrice de confusion (figure 1) confirme visuellement ces observations : les confusions principales apparaissent entre les paires (4,9), (3,5) et (2,7), avec un taux d'erreur global de 3,27 % sur le test. Une projection PCA des vecteurs d'entrée dans un plan 2D montre que certaines classes (0, 1) forment des groupes bien séparés alors que d'autres (4 et 9, 3 et 5) se recouvrent fortement, ce qui est cohérent avec la matrice de confusion. Ces observations mettent en évidence les limites des modèles denses : malgré un nombre de paramètres important, ils restent sensibles aux variations d'écriture et ne tirent pas parti de la structure spatiale des images.

## 2 Classification d'images naturelles : CIFAR-10

### 2.1 Du chiffre manuscrit à l'image naturelle

Sur MNIST, où chaque image est un chiffre isolé sur fond noir, parfaitement centré, un simple modèle linéaire atteint déjà 92 % d'exactitude, et un MLP à deux couches dépasse 96 % : le problème est en réalité presque résolu par des modèles denses, sans exploiter explicitement la structure 2D de l'image.

CIFAR-10 change radicalement la donne : les images représentent des *objets et animaux du monde réel* (avions, chats, voitures...), photographiés sous des angles variés, sur des fonds complexes, en couleur, et compressés à seulement  $32 \times 32$  pixels. Sur ce type d'images, un MLP atteint vite ses

limites : pour aller plus loin, il faut exploiter la **structure spatiale** via des couches *convolutives* (CNN).

## 2.2 Le dataset CIFAR-10

CIFAR-10 (Krizhevsky, 2009) se compose de 60 000 images RGB  $32 \times 32$ , réparties en 10 classes équilibrées (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion), avec un découpage standard 50 000/10 000 pour entraînement / test. Par rapport à MNIST, plusieurs difficultés s'ajoutent : **couleur** (3 canaux RGB, 3 072 entrées au lieu de 784), **variabilité intra-classe** (un oiseau peut être vu de face, de profil, sur fond bleu ou vert), **confusions sémantiques** (chat/chien, automobile/camion, cerf/cheval, avion/bateau), et **bruit de fond** (sujets enchâssés dans un environnement complexe). Compte tenu de la contrainte de calcul (CPU uniquement), nous avons travaillé sur un sous-ensemble de 10 000 images d'entraînement et 2 500 de test. Cela ne change rien aux conclusions qualitatives : les écarts structurels entre MLP et CNN restent parfaitement visibles.

## 2.3 Premier essai : reprendre le MLP de MNIST

Nous reproduisons d'abord la stratégie qui a fonctionné sur MNIST, en aplatissant l'image en un vecteur d'entrée de dimension 3 072 :  $3072 \rightarrow 256 \rightarrow 128 \rightarrow 10$ . Le décompte des paramètres donne  $786\,688 + 32\,896 + 1\,290 = \mathbf{820\,874}$  paramètres. C'est déjà beaucoup : presque 4 fois plus de paramètres qu'il n'y a de pixels d'entraînement au total. Le modèle a donc largement la capacité d'apprendre par cœur, c'est-à-dire de surapprendre.

Le MLP *met à plat* les pixels, ce qui détruit la structure 2D de l'image. Deux pixels voisins dans l'image se retrouvent éloignés dans le vecteur d'entrée, et il faut au MLP énormément de paramètres pour réapprendre cette adjacence. De plus, le MLP n'est pas *invariant par translation* : si l'objet est décalé de quelques pixels dans le cadre, le réseau voit une entrée complètement différente. Cette absence d'invariance est très pénalisante sur des images naturelles, où le sujet n'est jamais parfaitement centré.

## 2.4 Réseau convolutif (CNN) : l'idée clé

Une couche de convolution applique un **filtre** (matrice de petite taille, par exemple  $3 \times 3$ ) qui glisse sur l'image et produit en chaque position une réponse locale. Le même filtre est appliqué partout, avec trois conséquences fondamentales : **invariance par translation** (un motif détecté à une position est aussi détecté ailleurs), **partage des poids** (un filtre  $3 \times 3$  sur entrée RGB ne contient que 28 paramètres, applicable à n'importe quelle dimension d'image), et **hiérarchie des features** (la première couche détecte bords et couleurs, la deuxième des textures, la troisième des parties d'objets, la dernière l'objet entier).

Après chaque convolution, on applique une opération de *max-pooling*  $2 \times 2$  qui prend le maximum sur une petite fenêtre. Ce *downsampling* conserve les activations les plus fortes (motifs saillants), rend le réseau encore plus tolérant aux petits décalages, et réduit drastiquement la quantité de calcul des couches suivantes.

## 2.5 Architecture employée

Nous utilisons une architecture inspirée de LeNet (LeCun et al., 1998) adaptée à CIFAR-10 : trois blocs convolutifs ( $\text{Conv } 3 \rightarrow 32, 32 \rightarrow 64, 64 \rightarrow 64$ , chacun suivi de  $\text{ReLU} + \text{MaxPool } 2 \times 2$ ), puis Flatten (1024), Linear  $1024 \rightarrow 128 + \text{ReLU} + \text{Dropout}(0,25)$ , enfin Linear  $128 \rightarrow 10$  avec softmax via la CrossEntropyLoss. Le décompte des paramètres donne : conv1 896, conv2 18 496, conv3 36 928, fc1 131 200, fc2 1 290, soit 188 810 **paramètres au total**,  **$4,3\times$**  moins que le MLP. Un CNN bien conçu est à la fois *plus efficace* en nombre de paramètres *et* plus performant en généralisation qu'un MLP, parce que son architecture incarne déjà les bonnes *a priori* (localité, invariance par translation, hiérarchie) pour le domaine de la vision.

**Pourquoi 32, 64, 64 canaux ?** La progression  $32 \rightarrow 64 \rightarrow 64$  suit la même logique que les tailles  $128 \rightarrow 64$  du MLP MNIST : on augmente progressivement la profondeur (le nombre de filtres) à mesure que la résolution spatiale diminue. À chaque *max-pooling*, la taille spatiale est divisée par 4, donc on peut se permettre de doubler le nombre de filtres sans exploser le coût de calcul. Ce schéma « résolution spatiale qui diminue, profondeur en canaux qui augmente » est la signature de presque toutes les architectures CNN modernes (VGG, ResNet, EfficientNet).

**Pourquoi du Dropout sur la couche dense ?** La couche  $1024 \rightarrow 128$  est la plus grosse en paramètres (131 200) et la plus vulnérable au surapprentissage, parce qu'elle n'a pas la contrainte de partage des poids des convolutions. Le **Dropout(0.25)** désactive aléatoirement 25 % des neurones à chaque batch d'entraînement, ce qui force le réseau à ne pas trop s'appuyer sur un neurone unique et améliore mesurablement la généralisation.

## 2.6 Protocole expérimental

Optimiseur Adam ( $\text{lr} = 10^{-3}$ ), cross-entropy, batch 128, 8 époques (limite CPU), graine 42. Les pixels sont normalisés dans  $[-1, 1]$  via  $x \mapsto (x - 0,5)/0,5$ . Pour les images naturelles RGB, on préfère typiquement une normalisation centrée autour de 0 : cela accélère la convergence (gradients plus équilibrés) et fonctionne particulièrement bien avec ReLU et l'initialisation de Kaiming utilisée par défaut dans PyTorch. Sur MNIST,  $[0, 1]$  suffit car les pixels noirs/blancs sont déjà bien contrastés ; sur CIFAR, le centrage rend l'optimisation nettement plus stable.

## 2.7 Résultats

Le tableau 2 synthétise les résultats : le CNN améliore l'exactitude de plus de **10 points** avec presque **cinq fois moins** de paramètres que le MLP. Deux observations méritent d'être soulignées. **Le MLP sur-apprend** : l'écart entre accuracy d'entraînement (68,3 %) et de test (46,2 %) est de 22 points — symptôme typique du surapprentissage, le modèle mémorise les exemples sans apprendre des règles généralisables. **Le CNN généralise mieux** : l'écart train/test n'est plus que de 2,4 points (58,9 % vs 56,5 %). Le CNN apprend moins vite sur le train (il est plus contraint), mais cette contrainte est précisément ce qui le force à apprendre des features pertinentes plutôt que d'imprimer le training set. C'est l'incarnation parfaite du compromis biais-variance : on accepte une légère perte sur le train (plus de biais) en échange d'un énorme gain sur le test (moins de variance).

Les courbes d'apprentissage (figure 2) montrent les deux dynamiques : chez le MLP, train et

test divergent rapidement ; chez le CNN, elles restent collées l’une à l’autre, signe d’une bonne généralisation. La matrice de confusion (figure 3) montre une diagonale dense, mais quelques confusions systématiques entre classes sémantiquement proches : **chat** ↔ **chien** (silhouettes proches en basse résolution), **automobile** ↔ **camion**, **cerf** ↔ **cheval**, **avion** ↔ **bateau**. Ce sont des confusions *sémantiquement explicables* : le modèle se trompe sur les classes proches, comme le ferait un humain placé devant des vignettes  $32 \times 32$ . Les exemples qualitatifs (figure 4) confirment que les images mal classées correspondent souvent à des prises de vue ambiguës (sujet de profil, partiellement coupé, sur un fond inhabituel) ou à des sujets très petits dans le cadre.

## 2.8 Que voient les filtres convolutifs ?

Un atout pédagogique majeur des CNN est qu’on peut *visualiser* ce que la première couche a appris : chaque filtre  $3 \times 3 \times 3$  peut être affiché comme une petite image RGB (figure 5). On y reconnaît trois grandes catégories de détecteurs, exactement celles que la vision classique programme manuellement depuis des décennies : **détecteurs de bords orientés** (transitions sombre/clair horizontales, verticales et diagonales, qui rappellent les filtres de Sobel ou Prewitt), **détecteurs de couleur** (filtres uniformément rouges, bleus ou verts, pour repérer ciel bleu, herbe verte...), et **détecteurs de contraste local** (alternance fine de pixels clairs et sombres, qui répondent à des textures : poil, écailles, plumage). C’est exactement la base d’analyse que l’on construit manuellement en vision classique (filtres de Gabor, gradients orientés) : le CNN les apprend automatiquement à partir des données, sans qu’on ait à les spécifier.

**Bilan partie 2.** Sur des images naturelles, un MLP n’exploite pas la structure 2D et plafonne autour de 46 % d’exactitude malgré 820 k paramètres. Un CNN bien plus petit (189 k,  $4,3\times$  moins) généralise nettement mieux (56,5 %) grâce à trois *a priori* architecturaux : localité, partage des poids et hiérarchie. Nos résultats sous-estiment ce qu’un CNN peut faire : sur les 50 000 images complètes, avec un GPU et 30–50 époques, ce même CNN dépasse 75 % ; les architectures modernes (ResNet, EfficientNet) dépassent 95 % avec un peu de *data augmentation*. La leçon ne réside pas dans la valeur absolue, mais dans le saut qualitatif entre MLP et CNN sur les images naturelles.

## 3 Application médicale : détection de tumeurs mammaires

### 3.1 Du jeu d’école à l’enjeu clinique

Les deux premières parties ont posé la machinerie d’un classifieur d’images (modèle linéaire, MLP, CNN, cross-entropy, descente de gradient). Cette dernière partie déplace ces outils vers une **application médicale réelle** : la classification d’images mammaires en deux catégories, *malin* ou *normal/bénin*. Le changement de contexte est profond et impose de revoir plusieurs hypothèses : le coût d’une erreur n’est plus symétrique (**rater un cancer** est bien plus grave qu’alerter à tort), le dataset est très **petit** (quelques centaines d’images), les classes sont **déséquilibrées** (la classe minoritaire est précisément celle qu’il faut détecter), et des questions de **calibration**, **explicabilité** et **régulation** deviennent centrales avant tout déploiement. Cette partie est donc autant un exercice technique qu’un exercice de recul critique sur ce que peut et ne peut pas faire un modèle d’IA en santé.

### 3.2 Contexte et nuance sur le dataset

Le cancer du sein est le cancer le plus diagnostiqué chez la femme dans le monde. Le dépistage organisé repose sur deux modalités complémentaires : la **mammographie** (rayons X), gold standard du dépistage organisé (en France, examen biennal recommandé de 50 à 74 ans), et l'**échographie** mammaire, complémentaire en cas de tissu dense ou chez les femmes jeunes. Faute d'accès à un dataset libre de mammographies complet, nous travaillons sur **BreastMNIST** (Yang et al., 2023), qui contient *stricto sensu* des **échographies mammaires** et non des mammographies. La méthodologie reste identique (CNN, cross-entropy, sensibilité/spécificité), mais les conclusions quantitatives ne sont pas directement transposables au dépistage par rayons X.

### 3.3 Le dataset BreastMNIST

BreastMNIST est issu d'une cohorte de 780 images d'échographies mammaires, originellement  $500 \times 500$ , redimensionnées à  $64 \times 64$ . La tâche initiale (trois classes : normal, bénin, malin) est simplifiée en **classification binaire** : classe 0 = malin, classe 1 = normal ou bénin. Ce regroupement correspond à la décision clinique pertinente : ce qu'il faut absolument détecter, c'est la présence d'un cancer. Le split standard 7/1/2 donne 546 train / 78 validation / 156 test, avec  $\sim 27\%$  de cas malins partout (figure 6, tableau 3). Trois difficultés propres : **très petite taille** (546 exemples contre 50 000 pour CIFAR — risque élevé de surapprentissage) ; **déséquilibre** ( $\sim 27\%$  de malins, un modèle naïf qui prédirait toujours *bénin* obtient mécaniquement 73 % d'accuracy mais 0 % de sensibilité) ; **images bruitées** (peu de contraste, bruit *speckle* caractéristique de l'imagerie ultrasonore, lésions minuscules à  $64 \times 64$ ).

### 3.4 Métriques pertinentes en contexte médical

L'exactitude seule est trompeuse sur un dataset déséquilibré. Avec la convention *positif* = *malin*, on note VP (vrai positif), FN (faux négatif : cancer manqué), FP (faux positif : rappel inutile), VN. Les trois métriques principales sont :

$$\text{Accuracy} = \frac{VP+VN}{VP+VN+FP+FN}, \quad \text{Sensibilité} = \frac{VP}{VP+FN}, \quad \text{Spécificité} = \frac{VN}{VN+FP}.$$

En dépistage, on **privilégie systématiquement la sensibilité** : il est moins grave de rappeler une patiente pour un examen complémentaire (FP) que de la rassurer à tort avec un cancer méconnu (FN). Un faux négatif retarde le diagnostic de plusieurs mois, voire d'années entre deux examens, ce qui peut transformer un cancer curable en cancer métastatique. La sensibilité et la spécificité dépendent toutes deux du seuil (0,5 par défaut). Pour évaluer la qualité d'un modèle *indépendamment* du seuil, on utilise l'**AUC** (aire sous la courbe ROC) : un AUC de 0,5 correspond au hasard, 1,0 à une séparation parfaite, 0,8–0,9 à un modèle « bon » en pratique clinique. L'AUC mesure la probabilité qu'un malin tiré au hasard reçoive une probabilité prédite supérieure à celle d'un bénin tiré au hasard ; c'est devenu la métrique standard en imagerie médicale.

### 3.5 Modèle proposé

Étant donné la petite taille du dataset, nous évitons un modèle trop profond. L'architecture retenue : trois blocs convolutifs (Conv 1  $\rightarrow$  16, 16  $\rightarrow$  32, 32  $\rightarrow$  64, chacun + ReLU + MaxPool  $2 \times 2$ ), Flatten

(4096), Linear  $4096 \rightarrow 64 + \text{ReLU} + \text{Dropout}(0,5)$ , Linear  $64 \rightarrow 2$ . Soit 285 634 **paramètres**, dont la grosse couche dense  $4096 \rightarrow 64$  représente la plus grosse part. Le schéma suit exactement la logique de CIFAR : réduction progressive de la résolution ( $64 \rightarrow 32 \rightarrow 16 \rightarrow 8$ ) et augmentation parallèle du nombre de canaux ( $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$ ).

**Pourquoi un Dropout si agressif ?** On utilise un  $\text{Dropout}(0.5)$ , beaucoup plus élevé que le 0,25 de la partie 2. C’est la conséquence directe de la petite taille du dataset : avec 546 images d’entraînement, le risque de mémorisation est très fort. Désactiver aléatoirement la moitié des neurones de la couche dense à chaque batch force le réseau à apprendre des représentations redondantes et plus robustes.

**Gestion explicite du déséquilibre.** Nous pondérons la cross-entropy inversement à la fréquence de chaque classe :

$$w_{\text{malin}} = \frac{N}{2 \cdot N_{\text{malin}}} \approx 1,46, \quad w_{\text{bénin}} = \frac{N}{2 \cdot N_{\text{bénin}}} \approx 0,54,$$

où  $N$  est le nombre total d’exemples. Concrètement, chaque erreur sur un malin compte presque trois fois plus dans le gradient. Sans cette pondération, l’optimiseur ignore tout simplement la classe minoritaire : il trouve plus rentable de toujours prédire *bénin*.

**Régularisation complémentaire.** *Weight decay* de  $10^{-4}$  (régularisation  $L^2$  douce sur tous les poids) et *early stopping* : on évalue le modèle sur le set de validation à chaque époque et on conserve l’état qui obtient la meilleure performance *de validation*, pas le dernier état entraîné. Cela évite de surapprendre quand l’erreur train continue de baisser alors que la validation remonte.

### 3.6 Protocole expérimental

Adam ( $\text{lr} = 10^{-3}$ , *weight decay*  $10^{-4}$ ), batch 32 (plus petit que CIFAR pour compenser le faible nombre d’exemples), 30 époques avec *early stopping*, pixels normalisés dans  $[0, 1]$ , seuil par défaut 0,5 pour les métriques de confusion (l’AUC est calculée sur les probabilités brutes), graine 42.

### 3.7 Résultats

La figure 7 montre la convergence sur 30 époques : décroissance rapide pendant les 5 premières époques, puis plateau autour de 25 % d’erreur de validation, avec des oscillations. Ces oscillations sont inévitables avec un set de 78 images : chaque image mal classée fait varier l’accuracy de 1,3 point. L’*early stopping* permet de capturer le meilleur état du modèle malgré ce bruit. Les performances finales sur le test (156 images) sont résumées dans le tableau 4 : accuracy 83,3 %, AUC 0,856, sensibilité 59,5 %, spécificité 92,1 %, avec 25/42 vrais positifs, 17/42 faux négatifs, 105/114 vrais négatifs et 9/114 faux positifs.

**Lecture critique.** L’accuracy de 83 % est bonne en apparence, mais gonflée par la classe majoritaire (le baseline « toujours bénin » est déjà à 73 % : notre modèle apporte seulement  $\sim 10$  points). L’AUC de 0,856 est en revanche encourageant : le modèle classe correctement une grande partie des images selon leur probabilité d’être maligne — l’*information utile* est apprise, même si le

seuil par défaut ne l’exploite pas optimalement. Enfin, la sensibilité de 59,5 % est insuffisante en pratique clinique : le modèle rate 17 cancers sur 42, soit 40 % des cas. Ces faux négatifs sont le problème à traiter en priorité avant tout déploiement. La matrice de confusion (figure 8) confirme ces chiffres : très bon sur la majoritaire (105 bénins correctement classés sur 114), insuffisant sur la minoritaire. Les exemples qualitatifs (figure 9) montrent que les lésions malignes ratées présentent souvent un faible contraste avec les tissus environnants, ce qui les rend visuellement très proches d’images bénignes — y compris pour l’œil humain non expert.

### 3.8 Discussion : pourquoi la sensibilité est-elle modérée ?

Trois facteurs convergent. **Petite taille de l’entraînement** : 147 exemples malins seulement, ce qui limite ce que le modèle peut apprendre des variations morphologiques (taille, forme, texture, localisation). **Résolution faible** ( $64 \times 64$ ) : certaines lésions occupent quelques pixels seulement, à la limite du détectable ; une résolution plus fine aiderait mais le dataset n’en dispose pas. **Ambiguïté intrinsèque** : même un radiologue expérimenté ne pose pas de diagnostic certain à partir d’une seule échographie. Il croise plusieurs vues, l’âge, le contexte clinique, parfois une biopsie. Demander à un modèle de faire mieux à partir d’une seule image  $64 \times 64$  est intrinsèquement difficile.

### 3.9 Comment améliorer le modèle ?

**1. Ajuster le seuil de décision.** Le seuil 0,5 par défaut est arbitraire et ne reflète pas le compromis souhaité. En *baissant* le seuil (par exemple 0,3), on augmente la sensibilité au prix d’une spécificité plus faible. L’AUC élevé (0,856) indique que le modèle ordonne correctement les cas : un ajustement du seuil devrait permettre d’atteindre  $\sim 80\%$  de sensibilité pour  $\sim 75\%$  de spécificité.

**2. Data augmentation médicale.** Rotations légères ( $\pm 15^\circ$ ), *flips* horizontaux/verticaux, modifications de contraste/luminosité, *crops* aléatoires recentrés sur la lésion : autant de variations qui multiplient le dataset effectif par 5 à 10 sans introduire d’artefacts irréalistes.

**3. Transfer learning.** Initialiser le réseau avec un modèle pré-entraîné sur ImageNet (ResNet18, EfficientNet-B0) et faire un *fine-tuning*. Le réseau arrive déjà avec une connaissance générale de la vision (bords, textures, formes) et n’a qu’à spécialiser ses dernières couches. Gain typique : +5 à +10 points d’AUC.

**4. Plus de données et ensembles.** CBIS-DDSM ( $> 10\,000$  mammographies annotées) ou la compétition RSNA Breast Cancer Detection permettraient de changer d’échelle. L’*ensembling* (moyenner plusieurs CNN avec graines différentes) apporte typiquement +2 à +3 points de sensibilité.

### 3.10 Limites éthiques et cliniques

Au-delà de la performance brute, plusieurs aspects sont *indispensables* avant tout déploiement en milieu hospitalier. **Calibration** : un modèle peut être très bon en *ranking* (bon AUC) tout en produisant des probabilités mal calibrées ; pour le clinicien, un score « 80 % » doit correspondre à une vraie probabilité, pas à un nombre opaque. **Explicabilité** : un modèle qui dit « *cette image est maligne* » sans dire *où* il regarde ne sera pas adopté par les radiologues. Des techniques comme *Grad-CAM* visualisent les zones d’attention et permettent de vérifier que le modèle s’appuie sur

la lésion et non sur un artefact de bord ou un marquage d'appareil. **Validation externe** : un modèle qui obtient 90 % d'AUC dans un hôpital ne donne pas nécessairement les mêmes résultats ailleurs (problème du *domain shift* : machine, protocole, population). Toute validation sérieuse passe par un test sur données externes. **Régulation** : en France, tout dispositif médical d'IA doit obtenir un marquage CE médical et passer par l'évaluation de l'ANSM ; aux États-Unis, l'agrément FDA est requis. Le déploiement clinique est une démarche réglementaire de plusieurs années, qui exige des essais prospectifs en condition réelle. Enfin, **un outil d'aide, pas un remplaçant** : la doctrine actuelle est claire, l'IA *assiste* le radiologue mais ne le remplace pas. La décision finale reste médicale et humaine. L'IA peut prioriser les examens à risque, signaler les cas suspects, fournir une deuxième lecture, mais elle n'a pas la responsabilité du diagnostic.

### 3.11 Bilan général

Un CNN modeste atteint 83 % d'accuracy et un AUC de 0,86 sur BreastMNIST, ce qui est honorable pour 546 images d'entraînement seulement. Le déséquilibre de classe doit être traité explicitement (pondération de la cross-entropy), sous peine d'obtenir un modèle dégénéré. En contexte médical, l'accuracy ne suffit pas : sensibilité, spécificité et AUC sont indispensables. Notre modèle obtient 92 % de spécificité mais seulement 59,5 % de sensibilité ; le réglage du seuil et la data augmentation permettraient probablement de remonter ce chiffre à  $\sim 80$  % sans changer l'architecture. Toute la pipeline développée (softmax, cross-entropy, descente de gradient, CNN, pondération) est exactement la même que sur MNIST et CIFAR-10. C'est précisément cette généralité méthodologique qui rend ces techniques si puissantes : on adapte les détails (architecture, métriques, régularisation), mais le squelette reste universel. La principale leçon de ce projet n'est pas la valeur d'une métrique, mais la prise de conscience qu'un modèle d'IA médicale ne se résume pas à un chiffre : calibration, explicabilité, validation externe, régulation conditionnent autant le déploiement effectif en clinique que la performance brute.

## A Annexes — Figures, tableaux et code

### A.1 Tableaux récapitulatifs

| Modèle             | Architecture                                        | Paramètres | Erreur test |
|--------------------|-----------------------------------------------------|------------|-------------|
| Linéaire (softmax) | $784 \rightarrow 10$                                | 7 850      | 7,72 %      |
| MLP 1 couche       | $784 \rightarrow 128 \rightarrow 10$                | 101 770    | 3,46 %      |
| MLP 2 couches      | $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ | 109 386    | 3,27 %      |

TABLE 1 – Performances des modèles sur MNIST.

| Modèle                        | Paramètres     | Acc. train | Acc. test     |
|-------------------------------|----------------|------------|---------------|
| MLP (3072→256→128→10)         | 820 874        | 68,3 %     | 46,2 %        |
| <b>CNN (3 conv + 2 dense)</b> | <b>188 810</b> | 58,9 %     | <b>56,5 %</b> |

TABLE 2 – Comparaison MLP vs CNN sur CIFAR-10 (10 000 train / 2 500 test).

| Split        | Total | Malin | Normal/bénin | % malin |
|--------------|-------|-------|--------------|---------|
| Entraînement | 546   | 147   | 399          | 26,9 %  |
| Validation   | 78    | 21    | 57           | 26,9 %  |
| Test         | 156   | 42    | 114          | 26,9 %  |

TABLE 3 – Effectifs BreastMNIST par split (stratification conservée).

| Métrique                           | Valeur    |
|------------------------------------|-----------|
| Accuracy                           | 83,3 %    |
| AUC ROC                            | 0,856     |
| Sensibilité (rappel <i>malin</i> ) | 59,5 %    |
| Spécificité (rappel <i>bénin</i> ) | 92,1 %    |
| Vrais positifs (VP)                | 25 / 42   |
| Faux négatifs (FN)                 | 17 / 42   |
| Vrais négatifs (VN)                | 105 / 114 |
| Faux positifs (FP)                 | 9 / 114   |

TABLE 4 – Performances finales sur le test BreastMNIST (156 images).

## A.2 Figures MNIST

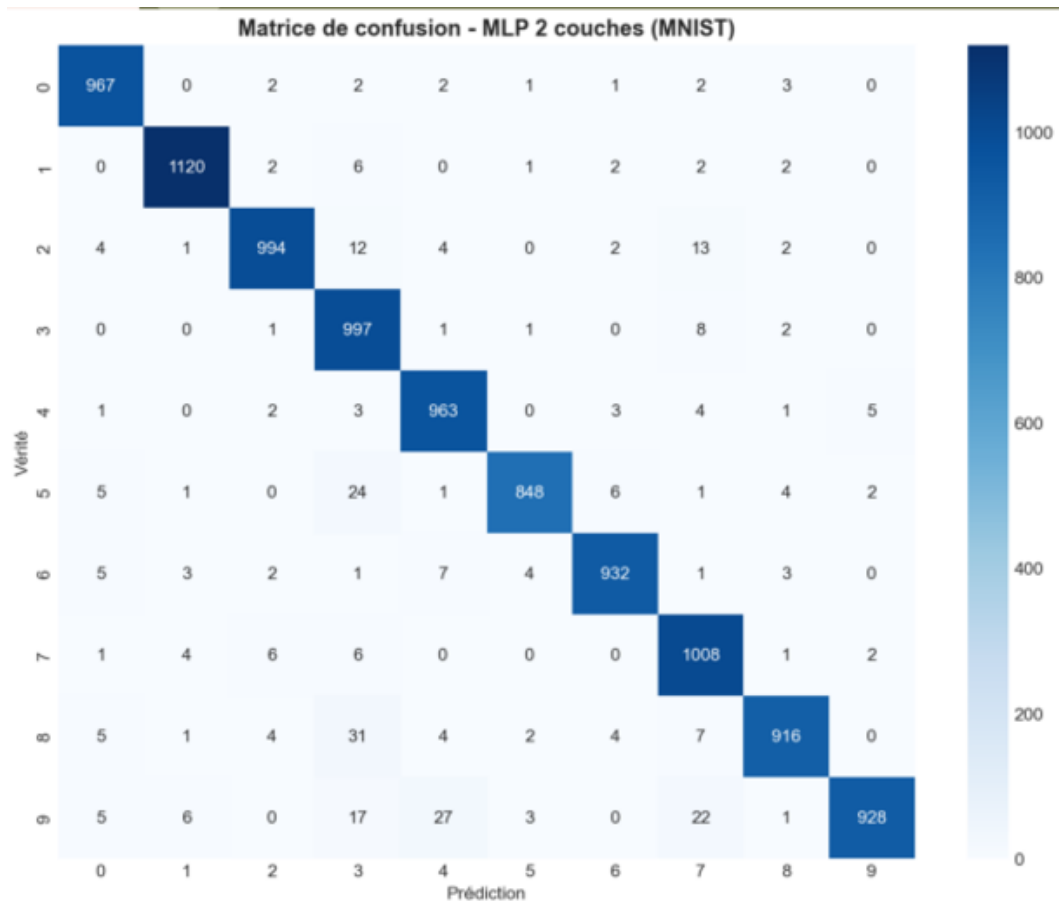


FIGURE 1 – Matrice de confusion du MLP à deux couches sur MNIST. La diagonale montre les bonnes classifications, les autres cases les erreurs entre classes.

### A.3 Figures CIFAR-10

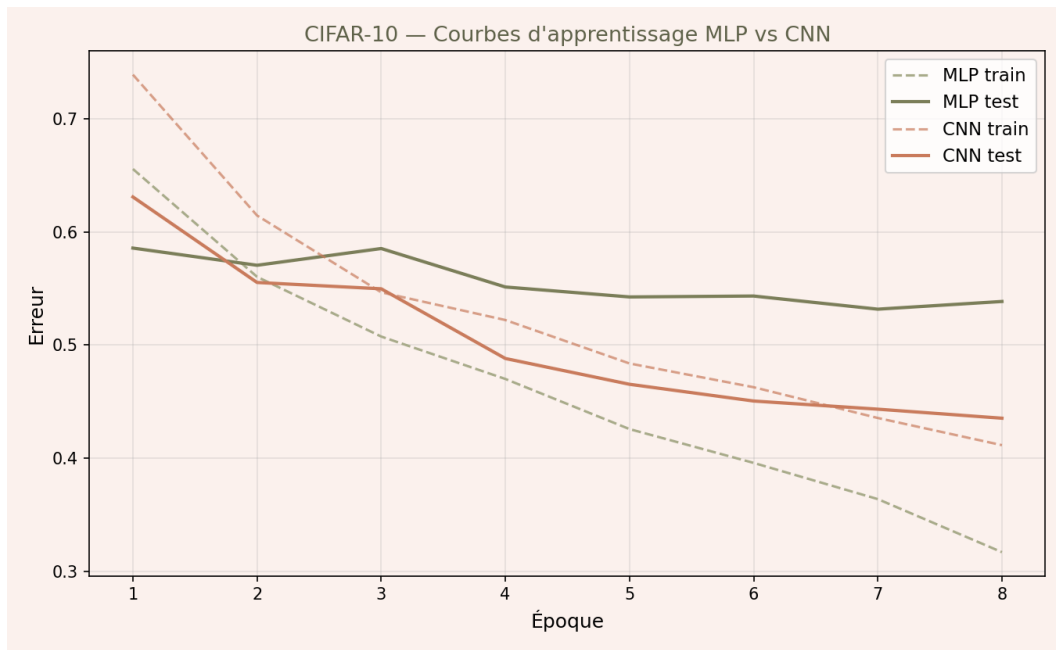


FIGURE 2 – Courbes d'apprentissage MLP vs CNN sur CIFAR-10. Pointillés : erreur d'entraînement ; trait plein : erreur de test.

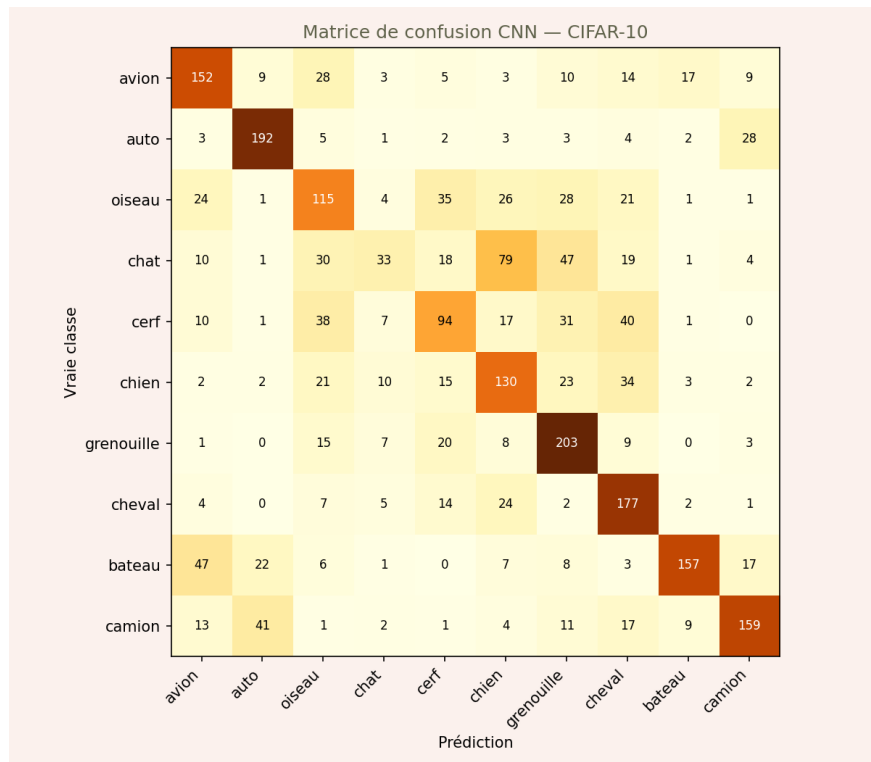


FIGURE 3 – Matrice de confusion du CNN sur le jeu de test CIFAR-10.



FIGURE 4 – Exemples CIFAR-10. En haut : prédictions correctes. En bas : prédictions erronées (étiquette vraie → prédite).



FIGURE 5 – Visualisation des 32 filtres de la première couche convolutive conv1. Chaque vignette représente un filtre  $3 \times 3 \times 3$  affiché comme une image RGB.

## A.4 Figures BreastMNIST

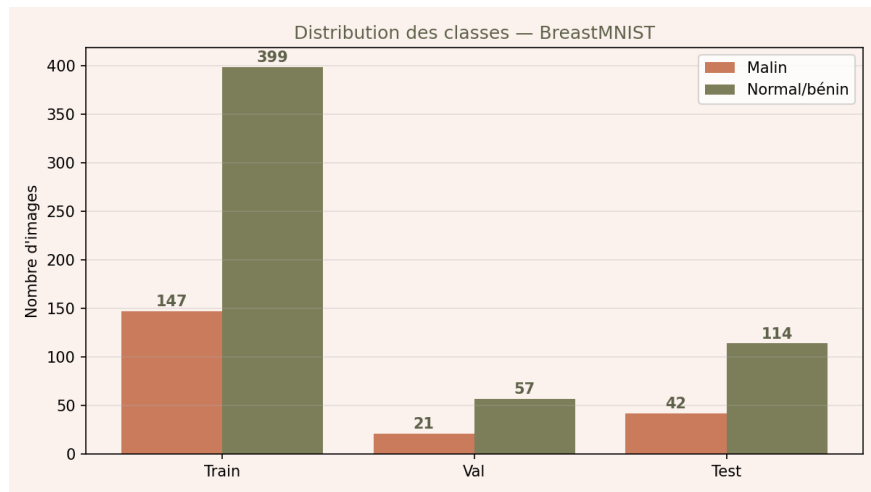


FIGURE 6 – Distribution des classes dans BreastMNIST. Le déséquilibre est important : environ 27 % d’images malignes seulement.

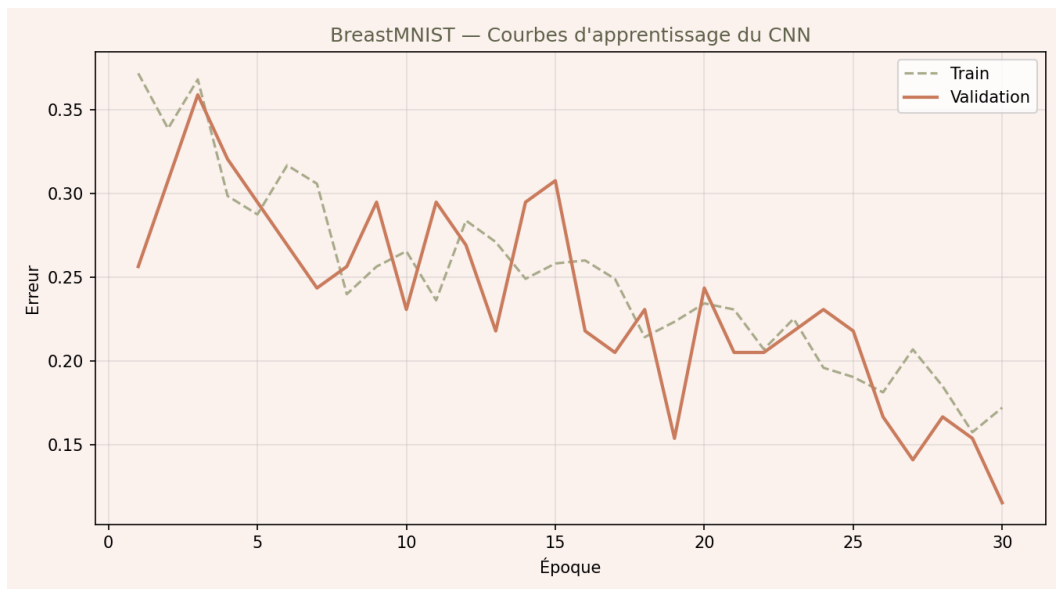


FIGURE 7 – Courbes d’apprentissage du CNN sur BreastMNIST. Les oscillations en validation sont typiques d’un petit dataset : avec 78 images de validation, chaque erreur représente 1,3 point d’accuracy.

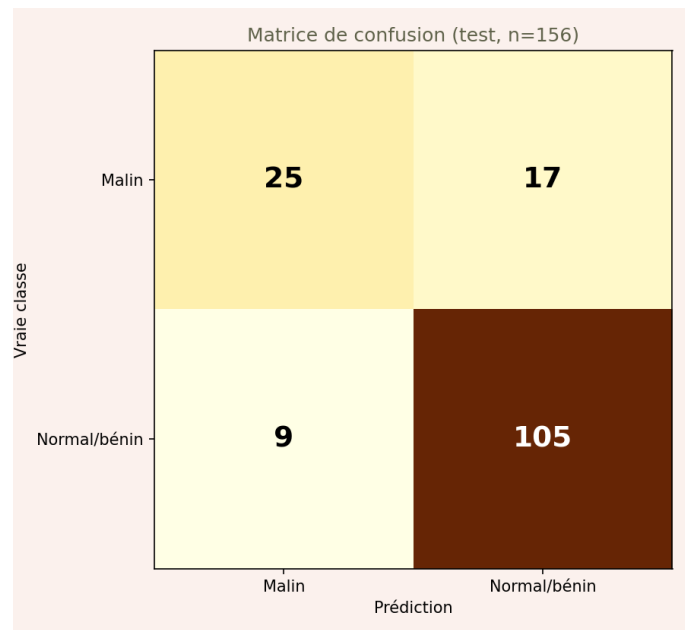


FIGURE 8 – Matrice de confusion sur le test BreastMNIST (156 images). Lignes : classe réelle ; colonnes : prédiction.

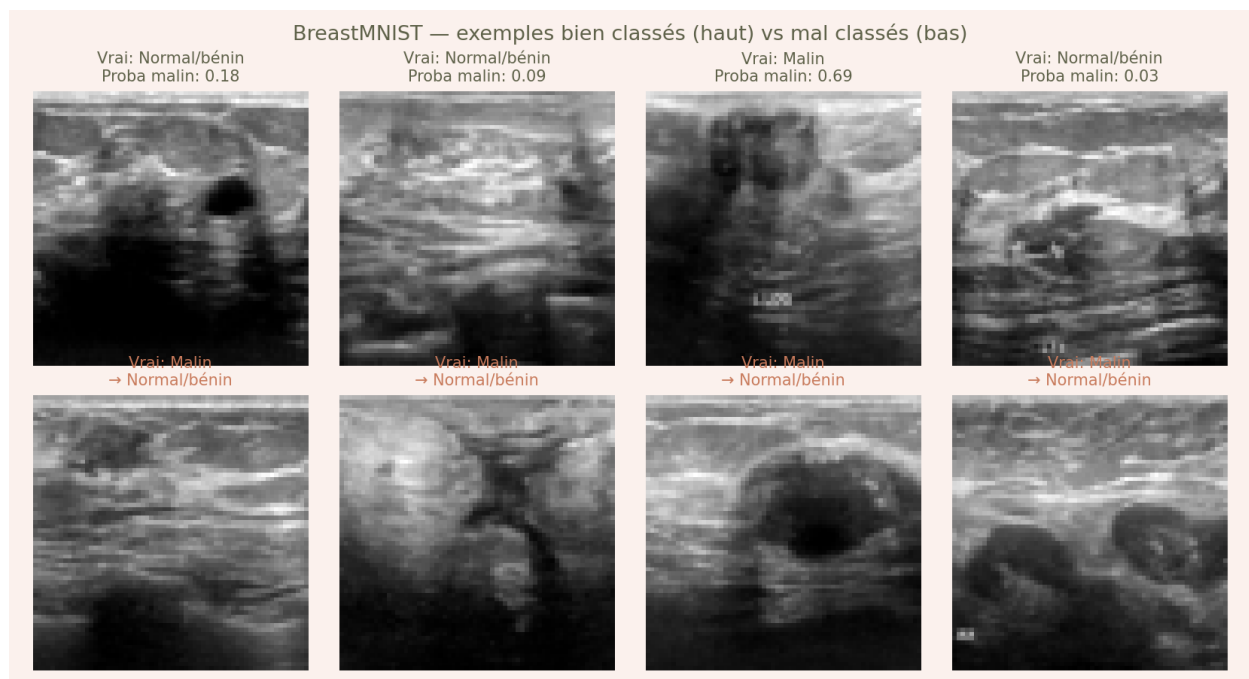


FIGURE 9 – Exemples BreastMNIST. En haut : prédictions correctes avec la probabilité estimée pour la classe malin. En bas : prédictions erronées (étiquette vraie / prédite).

## A.5 Extraits de code Python

```
class LinearClassifier:
    def __init__(self, d=784, K=10):
        self.A = np.random.randn(K, d) * np.sqrt(2.0 / d)
        self.b = np.zeros(K)
        self.history = {'loss_tr': [], 'loss_te': [], 'acc_tr': [], 'acc_te': []}

    4 usages
    def forward(self, X):
        return softmax(X @ self.A.T + self.b)
```

FIGURE 10 – Implémentation Python du calcul des scores  $o = Ax + b$  (méthode forward du modèle linéaire).

```
def softmax(o):
    o_shifted = o - np.max(o, axis=1, keepdims=True)
    exp_o = np.exp(o_shifted)
    return exp_o / np.sum(exp_o, axis=1, keepdims=True)
```

FIGURE 11 – Fonction softmax implémentée en Python.

```
def predict(self, X):
    return np.argmax(self.forward(X), axis=1)
```

FIGURE 12 – Calcul de la prédiction finale  $\hat{y}$  avec argmax (méthode predict).

```
def cross_entropy_loss(Y_true, P_pred, eps=1e-12):
    n = Y_true.shape[0]
    return -np.sum(Y_true * np.log(np.clip(P_pred, eps, 1 - eps))) / n
```

FIGURE 13 – Implémentation de la fonction de coût d'entropie croisée (log loss) en Python.

```

for s in range(0, n, bs):
    e = min(s + bs, n)
    Xb, Yb = Xtr[idx[s:e]], Ytr[idx[s:e]]
    Pb = self.forward(Xb)
    delta = Pb - Yb

    self.A -= lr * (delta.T @ Xb) / (e - s)
    self.b -= lr * np.mean(delta, axis=0)

```

FIGURE 14 – Boucle d’entraînement mini-batch et mise à jour des paramètres  $A$  et  $\vec{b}$  via  $(P - Y)$ .

```

def accuracy(y_true, y_pred):
    return np.mean(y_true == y_pred)

```

FIGURE 15 – Calcul de l’accuracy et du taux d’erreur  $1 - \text{acc}$  dans le code d’entraînement.

```

def forward(self, X):
    self.z = [X]
    self.o = []
    cur = X

    for i in range(self.L):
        oi = cur @ self.W[i].T + self.b[i]
        self.o.append(oi)
        cur = relu(oi) if i < self.L - 1 else softmax(oi)
        self.z.append(cur)

    return cur

```

FIGURE 16 – Propagation avant dans le MLP (méthode `forward`). Chaque couche cachée applique une transformation linéaire suivie de l’activation ReLU ; la dernière couche utilise softmax.

```

def backward(self, Y, lr):
    n = Y.shape[0]
    delta = self.z[-1] - Y

    for i in range(self.L - 1, -1, -1):
        dW = (delta.T @ self.z[i]) / n
        db = np.mean(delta, axis=0)

        if i > 0:
            delta = (delta @ self.W[i]) * relu_derivative(self.o[i-1])

        self.W[i] -= lr * dW
        self.b[i] -= lr * db

```

FIGURE 17 – Implémentation de la rétropropagation dans le perceptron multi-couches (classe `MLPClassifier`).

## B Références

- [LeCun 1998] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 1998.
- [Krizhevsky 2009] Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto, 2009.
- [Goodfellow 2016] Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. MIT Press, 2016.
- [He 2016] He, K., Zhang, X., Ren, S., & Sun, J. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
- [Selvaraju 2017] Selvaraju, R. R., Cogswell, M., Das, A., et al. Grad-CAM : Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*, 2017.
- [Yang 2023] Yang, J., Shi, R., Wei, D., et al. MedMNIST v2 : a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10, 2023.
- [HAS 2024] Haute Autorité de Santé. *Dépistage du cancer du sein en France : recommandations actualisées*, 2024.